

LAPORAN PENGUJIAN WHITE BOX TESTING

Metode Basic Path Testing – Modul Checkout ACID



Disusun Oleh:

M. Irvan Alfiansyah	20231310046
M. Nur Yanfa	20231310047

**PRODI TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER DAN SISTEM INFORMASI
UNIVERSITAS KEBANGSAAN REPUBLIK INDONESIA
2026**

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, kami dapat menyelesaikan laporan pengujian yang berjudul "Laporan Pengujian White Box Testing – Metode Basic Path Testing Modul Checkout ACID". Laporan ini disusun untuk memenuhi tugas mata kuliah Pengujian Perangkat Lunak.

Kami menyadari bahwa penyelesaian laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, kami ingin menyampaikan ucapan terima kasih yang sebesar-besarnya kepada dosen pengampu mata kuliah Pengujian Perangkat Lunak yang telah memberikan bimbingan, ilmu, serta arahan yang sangat berharga selama proses perkuliahan.

Dalam laporan ini, dibahas mengenai pengujian White Box dengan metode Basic Path Testing pada modul Checkout ACID dari sistem aplikasi e-commerce UMKM Sayuran berbasis Go. Pengujian mencakup analisis Cyclomatic Complexity, identifikasi independent path, perancangan test case, hingga eksekusi pengujian pada lima fungsi utama yang meliputi `classifyCheckoutError`, `OrderHandler.Checkout`, `OrderHandler.GetMyOrders`, `orderUsecase.Checkout`, dan `CheckoutTransaction (Repository ACID)`.

Kami menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca mengenai pentingnya pengujian white box dalam memastikan kualitas dan keandalan perangkat lunak.

Bandung, 03 Juni 2026

DAFTAR ISI

KATA PENGANTAR	2
DAFTAR ISI.....	3
BAB I BAB I PENDAHULUAN	5
1.1 Latar Belakang	5
1.2 Rumusan Masalah.....	5
1.3 Tujuan Penulisan.....	5
1.4 Batasan Masalah	6
BAB II BAB II LANDASAN TEORI	7
2.1 White Box Testing	7
2.2 Basic Path Testing.....	7
2.3 Cyclomatic Complexity	8
BAB III BAB III RUANG LINGKUP PENGUJIAN	9
3.1 Deskripsi Sistem yang Diuji	9
3.2 Fungsi yang Diuji.....	9
3.3 Batasan Scope Pengujian	10
BAB IV BAB IV ANALISIS DAN HASIL PENGUJIAN	11
4.1 Pengujian Fungsi classifyCheckoutError.....	11
4.1.1 Deskripsi Fungsi	11
4.1.2 Source Code yang Diuji	11
4.1.3 Node Flow Graph.....	11
4.1.4 Cyclomatic Complexity	13
4.1.5 Independent Path dan Test Case	13
4.2 Pengujian Fungsi OrderHandler.Checkout	13
4.2.1 Deskripsi Fungsi	13
4.2.2 Source Code	13
4.2.3 Node Flow Graph.....	14
4.2.4 Cyclomatic Complexity	16
4.2.5 Independent Path dan Test Case	16
4.3 Pengujian Fungsi OrderHandler.GetMyOrders	16
4.3.1 Deskripsi Fungsi	16
4.3.2 Source Code	16
4.3.3 Node Flow Graph.....	17
4.3.4 Cyclomatic Complexity	19
4.3.5 Independent Path dan Test Case	19

4.4	Pengujian Fungsi orderUsecase.Checkout.....	19
4.4.1	Deskripsi Fungsi	19
4.4.2	Source Code	19
4.4.3	Node Flow Graph.....	20
4.4.4	Cyclomatic Complexity	22
4.4.5	Independent Path dan Test Case	22
4.5	Pengujian Fungsi CheckoutTransaction (Repository ACID)	22
4.5.1	Deskripsi Fungsi	22
4.5.2	Source Code	23
4.5.3	Node Flow Graph.....	24
4.5.4	Cyclomatic Complexity	25
4.5.5	Independent Path dan Test Case	26
BAB V BAB V REKAPITULASI DAN KESIMPULAN		28
5.1	Rekapitulasi Cyclomatic Complexity	28
5.2	Mapping White-Box ke Black-Box	28
5.3	Hasil Eksekusi Pengujian.....	28
5.3.1	Ringkasan Hasil	28
5.3.2	Perintah Verifikasi	29
5.3.3	File Test yang Digunakan	29
5.4	Kesimpulan	29
DAFTAR PUSTAKA		31

BAB I BAB I PENDAHULUAN

1.1 Latar Belakang

Perkembangan aplikasi berbasis web dan layanan e-commerce yang semakin pesat menuntut adanya proses pengujian perangkat lunak yang komprehensif dan terstruktur. Pengujian yang baik tidak hanya memastikan bahwa sistem berjalan sesuai fungsionalitas yang diharapkan dari sudut pandang pengguna, tetapi juga memvalidasi logika internal program secara menyeluruh agar setiap jalur eksekusi telah teruji dan bebas dari cacat tersembunyi.

Sistem e-commerce UMKM Sayuran yang dikembangkan oleh Kelompok kami menggunakan bahasa pemrograman Go (Golang) dengan arsitektur berlapis yang terdiri atas Handler, Usecase, dan Repository. Modul Checkout merupakan inti dari sistem transaksi yang mengelola alur pembelian mulai dari validasi sesi pengguna, pemrosesan keranjang belanja, pengurangan stok, pembuatan order, hingga integrasi dengan payment gateway Midtrans. Mengingat kompleksitas dan kekritisian modul ini, diperlukan pengujian White Box yang menyeluruh untuk memastikan setiap jalur logika berjalan dengan benar dan seluruh properti ACID (Atomicity, Consistency, Isolation, Durability) terjaga pada lapisan repository.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam laporan ini adalah sebagai berikut:

1. Berapa nilai Cyclomatic Complexity ($V(G)$) dari setiap fungsi pada modul Checkout ACID?
2. Apa sajakah independent path yang harus diuji pada setiap fungsi tersebut?
3. Apakah seluruh independent path aktif telah diuji dan lulus (PASS) pengujian?
4. Bagaimana pemetaan test case White Box terhadap skenario Black Box yang telah ditetapkan sebelumnya?

1.3 Tujuan Penulisan

Laporan ini disusun dengan tujuan sebagai berikut:

1. Menghitung dan mendokumentasikan nilai Cyclomatic Complexity setiap fungsi pada modul Checkout ACID.

2. Mengidentifikasi seluruh independent path aktif dan merancang test case yang sesuai.
3. Membuktikan bahwa coverage path aktif mencapai 100% melalui eksekusi pengujian.
4. Menyajikan pemetaan antara pengujian White Box dan Black Box sebagai validasi silang (cross-validation).

1.4 Batasan Masalah

Agar pembahasan dalam laporan ini lebih fokus dan terarah, ditetapkan batasan masalah sebagai berikut:

1. Pengujian hanya mencakup fungsi aktif pada modul Checkout ACID, yaitu: `classifyCheckoutError`, `OrderHandler.Checkout`, `OrderHandler.GetMyOrders`, `orderUsecase.Checkout`, dan `CheckoutTransaction`.
2. Fitur voucher tidak dimasukkan dalam scope pengujian karena tidak digunakan pada implementasi Checkout ACID aktif, sehingga path voucher tidak dihitung sebagai independent path aktif.
3. Referensi skenario Black Box diambil dari dokumen EC-BVA-TC Kel 5 Ver-2.

BAB II BAB II LANDASAN TEORI

2.1 White Box Testing

White Box Testing adalah teknik pengujian perangkat lunak yang berfokus pada struktur internal dan logika program. Berbeda dengan Black Box Testing yang hanya menguji fungsionalitas dari sudut pandang pengguna tanpa memperhatikan implementasi, White Box Testing memerlukan pengetahuan tentang kode sumber agar dapat merancang test case yang menguji jalur-jalur eksekusi, percabangan kondisional, dan perulangan secara langsung.

Tujuan utama White Box Testing adalah memastikan bahwa setiap baris kode, setiap cabang keputusan, dan setiap jalur logika telah dieksekusi minimal satu kali selama pengujian. Hal ini memungkinkan identifikasi bug yang tidak terlihat dari antarmuka luar, seperti logika kondisional yang salah, dead code, atau edge case yang tidak tertangani.

2.2 Basic Path Testing

Basic Path Testing adalah salah satu teknik White Box Testing yang diperkenalkan oleh Tom McCabe pada tahun 1976. Teknik ini menjamin bahwa setiap jalur logika independen (independent path) dalam suatu program dieksekusi minimal satu kali selama pengujian. Sebuah independent path didefinisikan sebagai jalur eksekusi yang melewati setidaknya satu edge atau kondisi baru yang belum dilewati oleh jalur sebelumnya.

Langkah-langkah penerapan Basic Path Testing adalah sebagai berikut:

1. Menggambar atau menganalisis Control Flow Graph (CFG) dari kode sumber.
2. Menghitung nilai Cyclomatic Complexity ($V(G)$) untuk menentukan jumlah minimum independent path.
3. Mengidentifikasi himpunan basis path (basis set) yang mencakup semua predicate node.
4. Merancang test case untuk setiap independent path dalam basis set.
5. Mengeksekusi test case dan mendokumentasikan hasilnya.

2.3 Cyclomatic Complexity

Cyclomatic Complexity ($V(G)$) adalah metrik kuantitatif yang mengukur kompleksitas logika suatu program berdasarkan jumlah jalur independen yang dapat dilalui dalam alur kontrolnya. Metrik ini dikembangkan oleh Thomas J. McCabe Sr. (1976) dan menjadi standar yang banyak digunakan dalam dunia rekayasa perangkat lunak.

Rumus penghitungan Cyclomatic Complexity menggunakan pendekatan Predicate Node adalah:

$$V(G) = P + 1$$

Simbol	Keterangan
$V(G)$	Cyclomatic Complexity / jumlah jalur independen minimum
P	Jumlah predicate node / percabangan

Di mana P adalah jumlah predicate node (simpul keputusan), yaitu setiap titik dalam alur kontrol yang memiliki dua atau lebih cabang keluar (IF, SWITCH, WHILE, FOR, dan sejenisnya).

Nilai $V(G)$	Kategori Risiko
1 – 10	Rendah – program sederhana, mudah diuji
11 – 20	Sedang – perlu perhatian ekstra dalam pengujian
21 – 50	Tinggi – program kompleks, risiko bug meningkat
> 50	Sangat Tinggi – tidak dapat diuji, perlu refaktor

Tabel 1. Kategori Risiko Berdasarkan Nilai Cyclomatic Complexity

BAB III BAB III RUANG LINGKUP PENGUJIAN

3.1 Deskripsi Sistem yang Diuji

Sistem yang diuji adalah aplikasi e-commerce UMKM Sayuran yang dikembangkan menggunakan bahasa pemrograman Go (Golang) dengan framework Gin dan ORM GORM. Sistem ini mengimplementasikan arsitektur berlapis yang terdiri atas tiga lapisan utama: Handler (Delivery Layer), Usecase (Business Logic Layer), dan Repository (Data Access Layer).

Modul Checkout mengimplementasikan pola transaksi ACID menggunakan fitur `db.Transaction()` dari GORM, yang memastikan bahwa seluruh operasi database dalam satu proses checkout (pengurangan stok, pembuatan order, penghapusan cart) berjalan secara atomis. Sistem juga terintegrasi dengan payment gateway Midtrans Snap API untuk pembuatan token pembayaran.

Nama Sistem	Aplikasi E-Commerce UMKM Sayuran
Jenis Pengujian	White Box Testing
Metode	Basic Path Testing / Cyclomatic Complexity
Modul yang Diuji	Checkout & Transaksi ACID
Referensi Black-Box	Dokumen_EC-BVA-TC_Kel_5_Ver-2.pdf
File Utama	backend-go/internal/delivery/http/order_handler.go
Tanggal Pengujian	03 Juni 2026
Penguji	Kelompok 5

Tabel 2. Identitas Pengujian

3.2 Fungsi yang Diuji

No	Fungsi	File	Layer
1	<code>classifyCheckoutError</code>	<code>order_handler.go</code>	Handler helper
2	<code>OrderHandler.Checkout</code>	<code>order_handler.go</code>	Handler
3	<code>OrderHandler.GetMyOrders</code>	<code>order_handler.go</code>	Handler
4	<code>orderUsecase.Checkout</code>	<code>order_usecase.go</code>	Usecase
5	<code>CheckoutTransaction</code>	<code>order_repository.go</code>	Repository

Tabel 3. Daftar Fungsi yang Diuji

3.3 Batasan Scope Pengujian

Pengujian ini hanya mencakup fitur aktif Checkout ACID. Fitur voucher tidak digunakan dalam scope pengujian ini meskipun parameter voucherCode masih terdapat pada beberapa signature kode untuk kompatibilitas. CheckoutTransaction pada implementasi aktif tidak menjalankan validasi atau diskon voucher. Oleh karena itu, path voucher tidak dihitung sebagai independent path aktif dan tidak dimasukkan ke denominator coverage.

ID Black-Box	Skenario	Dicakup White-Box
CHK-01	Checkout berhasil	Ya
CHK-02	Checkout stok tidak mencukupi	Ya
CHK-03	Race condition checkout bersamaan	Ya
CHK-04	Checkout keranjang kosong	Ya
CHK-05	Midtrans API error	Ya, dengan mock snapTokenFn
CHK-06	Checkout tanpa JWT/session	Ya
CHK-07	Validasi atomicity stok	Ya
CHK-08	Status pesanan PENDING	Ya

Tabel 4. Pemetaan Skenario Black-Box ke White-Box

BAB IV BAB IV ANALISIS DAN HASIL PENGUJIAN

4.1 Pengujian Fungsi `classifyCheckoutError`

File: `order_handler.go`

4.1.1 Deskripsi Fungsi

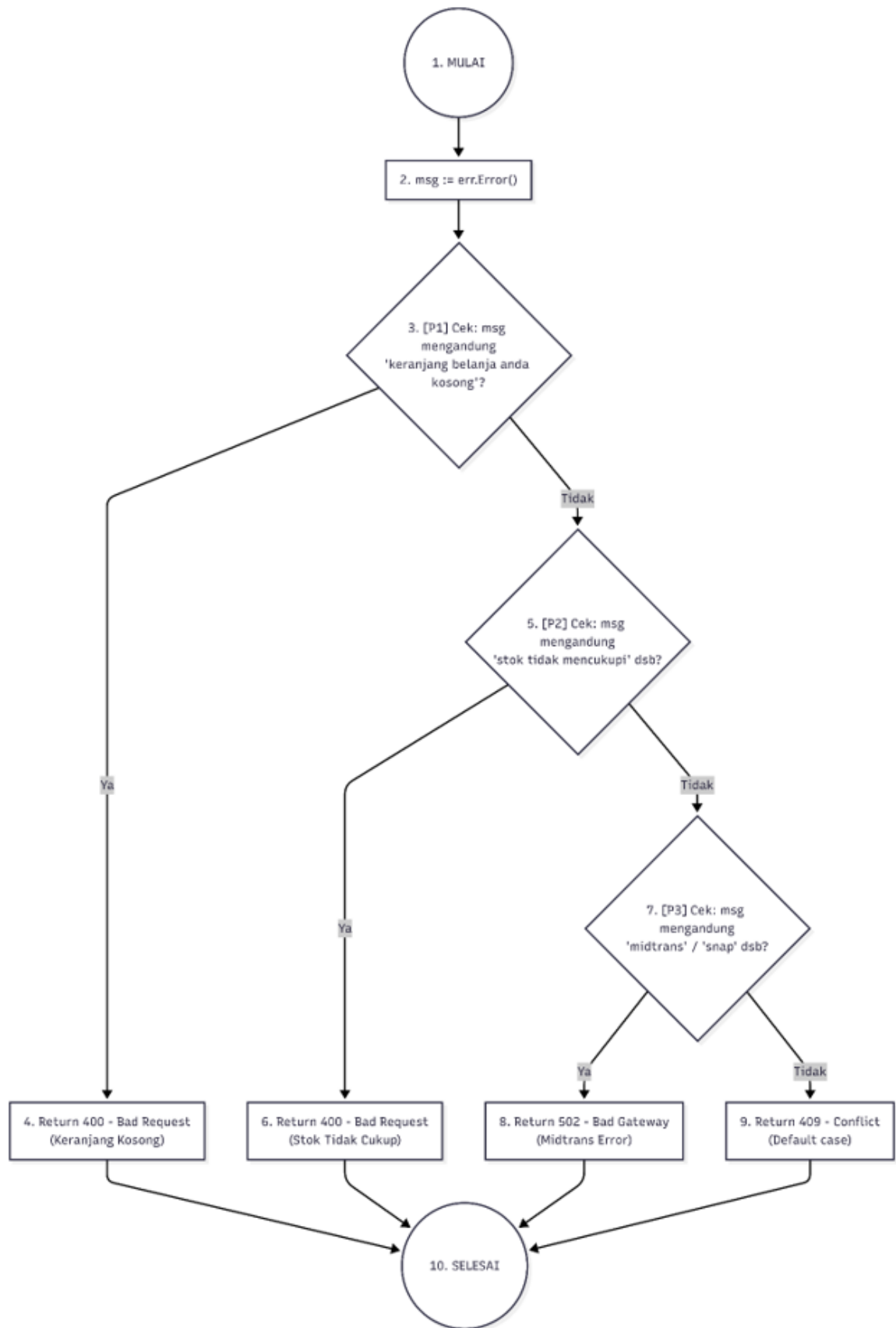
Fungsi helper yang mengklasifikasikan error dari proses checkout menjadi HTTP status code yang sesuai. Menerima satu parameter error dan mengembalikan `AppError` dengan kode HTTP 400, 502, atau 409.

4.1.2 Source Code yang Diuji

```
func classifyCheckoutError(err error) *response.AppError {
    msg := err.Error()
    switch {
    case strings.Contains(msg, "keranjang belanja anda kosong"):
        return response.ErrBadRequest(msg)
    case strings.Contains(msg, "stok tidak mencukupi"),
        strings.Contains(msg, "Stok"),
        strings.Contains(msg, "stok"):
        return response.ErrBadRequest(msg)
    case strings.Contains(msg, "midtrans"),
        strings.Contains(msg, "Midtrans"),
        strings.Contains(msg, "snap"),
        strings.Contains(msg, "payment gateway"):
        return &response.AppError{Code: http.StatusBadGateway, ...}
    default:
        return response.ErrConflict(msg)
    }
}
```

4.1.3 Node Flow Graph

Node Flow Graph – `classifyCheckoutError`



4.1.4 Cyclomatic Complexity

$$V(G) = P + 1 = 3 + 1 = 4$$

Terdapat 3 predicate node (P1, P2, P3), sehingga menghasilkan 4 independent path yang harus diuji.

#	Predicate Node	Kondisi
P1	Node 3	strings.Contains(msg, "keranjang belanja anda kosong")
P2	Node 5	Contains("stok tidak mencukupi") Contains("Stok") Contains("stok")
P3	Node 7	Contains("midtrans") Contains("Midtrans") Contains("snap") Contains("payment gateway")

Tabel 5. Predicate Node – *classifyCheckoutError*

4.1.5 Independent Path dan Test Case

ID	BB	Path	Input	Expected	Status
WB-CE-01	CHK-04	CE-1	Error cart kosong	HTTP 400	PASS
WB-CE-02	CHK-02	CE-2	Error stok tidak cukup	HTTP 400	PASS
WB-CE-03	CHK-05	CE-3	Error Midtrans/snap	HTTP 502	PASS
WB-CE-04	Tambahan	CE-4	Error lain (default)	HTTP 409	PASS

Tabel 6. Test Case – *classifyCheckoutError*

4.2 Pengujian Fungsi OrderHandler.Checkout

File: order_handler.go (baris 60-87)

4.2.1 Deskripsi Fungsi

Handler HTTP untuk endpoint POST /api/v1/orders/checkout. Memvalidasi JWT, mengekstrak user_id, memanggil usecase Checkout, dan mengembalikan response sesuai hasil. Field VoucherCode masih dibaca dari request, tetapi pada scope Checkout ACID aktif tidak ada proses voucher di repository.

4.2.2 Source Code

```
func (h *OrderHandler) Checkout(c *gin.Context) {  
    uidVal, exists := c.Get("user_id")  
    if !exists {  
        response.Error(c, response.ErrUnauthorized("Sesi pengguna  
tidak valid"))  
        return  
    }  
}
```

```

uid, ok := uidVal.(string)
if !ok {
    response.Error(c, response.ErrUnauthorized("Sesi pengguna
tidak valid"))
    return
}

var req struct {
    VoucherCode string `json:"voucher_code"`
}
_ = c.ShouldBindJSON(&req)

order, err := h.orderUsecase.Checkout(uid, req.VoucherCode)
if err != nil {
    response.Error(c, classifyCheckoutError(err))
    return
}

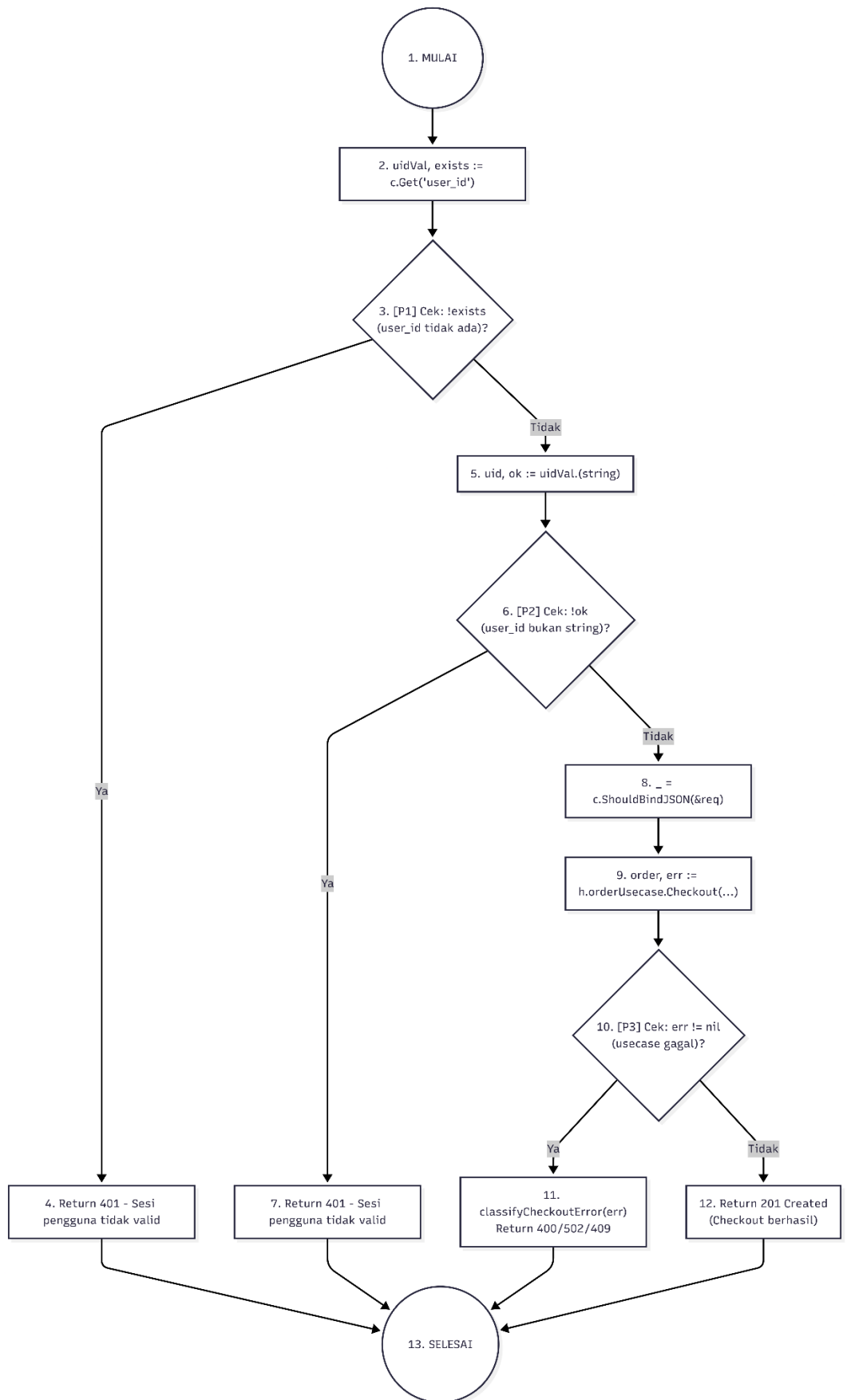
response.Success(c, http.StatusCreated,
    "Checkout berhasil. Silakan lakukan pembayaran.", order)
}

```

Catatan: field VoucherCode masih dibaca dari request, tetapi pada scope Checkout ACID aktif tidak ada proses voucher di repository.

4.2.3 Node Flow Graph

Node Flow Graph - OrderHendler.Checkout



4.2.4 Cyclomatic Complexity

$$V(G) = P + 1 = 3 + 1 = 4$$

#	Predicate Node	Kondisi
P1	Node 3	!exists — user_id tidak ada di context JWT
P2	Node 6	!ok — user_id ada tapi bukan tipe string
P3	Node 10	err != nil — usecase Checkout mengembalikan error

Tabel 7. Predicate Node – OrderHandler.Checkout

4.2.5 Independent Path dan Test Case

ID	BB	Path	Skenario	Expected	Status
WB-CHK-H01	CHK-06	H-1	user_id tidak ada	HTTP 401	PASS
WB-CHK-H02	CHK-06	H-2	user_id bukan string	HTTP 401	PASS
WB-CHK-H03	CHK-04/02	H-3	Usecase checkout error	HTTP 400/502/409	PASS
WB-CHK-H04	CHK-01	H-4	Checkout berhasil	HTTP 201 Created	PASS

Tabel 8. Test Case – OrderHandler.Checkout

4.3 Pengujian Fungsi OrderHandler.GetMyOrders

File: order_handler.go (baris 134-153)

4.3.1 Deskripsi Fungsi

Handler HTTP untuk endpoint GET /api/v1/orders/my. Memvalidasi JWT, mengekstrak user_id, memanggil usecase GetMyOrders, dan mengembalikan daftar order milik pengguna yang terautentikasi.

4.3.2 Source Code

```
func (h *OrderHandler) GetMyOrders(c *gin.Context) {
    uidVal, exists := c.Get("user_id")
    if !exists {
        response.Error(c, response.ErrUnauthorized("Sesi pengguna tidak valid"))
        return
    }
    uid, ok := uidVal.(string)
    if !ok {
        response.Error(c, response.ErrUnauthorized("Sesi pengguna tidak valid"))
        return
    }

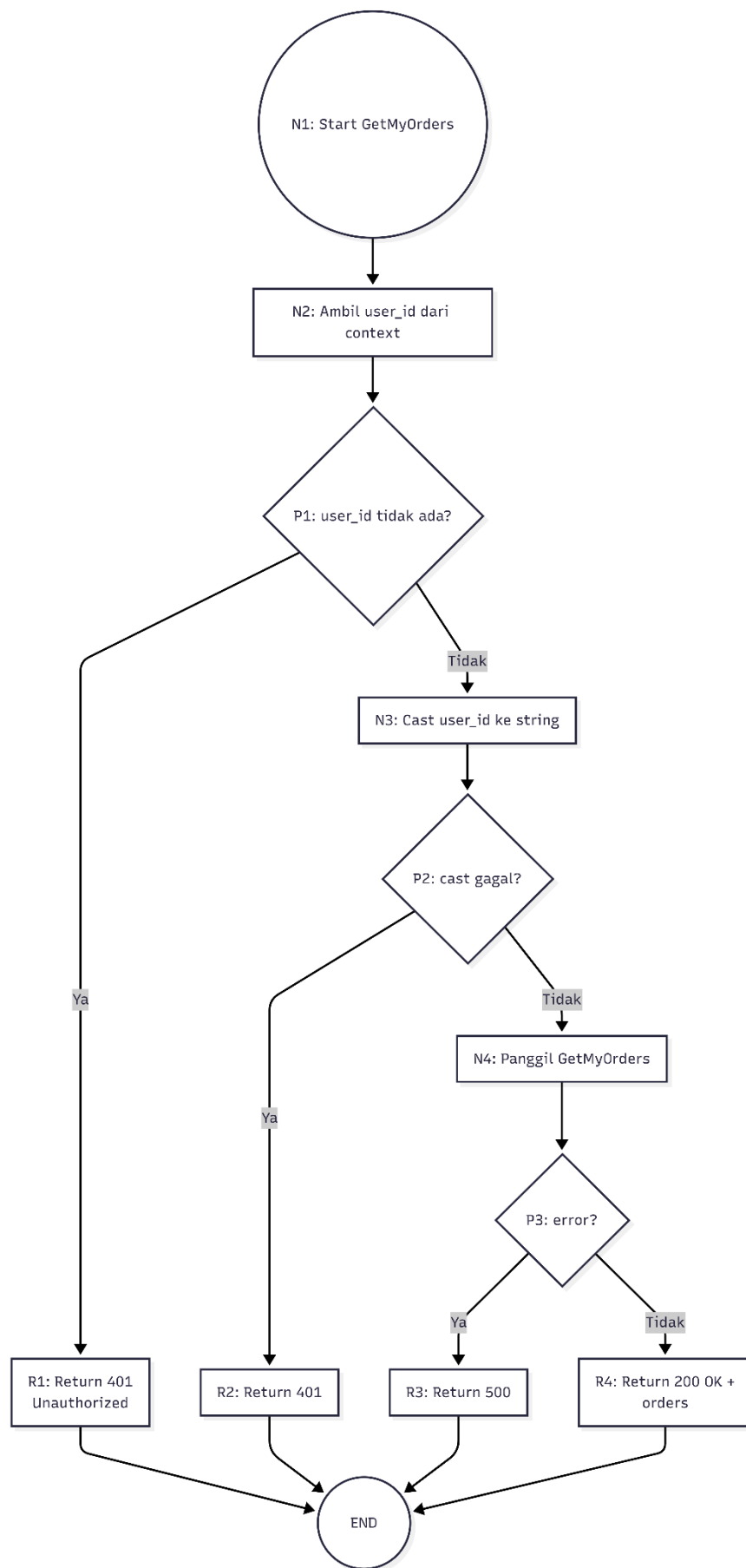
    orders, err := h.orderUsecase.GetMyOrders(uid)
    if err != nil {
        response.Error(c, response.ErrInternal(err.Error()))
    }
}
```



```
        return  
    }  
  
    response.Success(c, http.StatusOK, "Berhasil memuat pesanan",  
orders)  
}
```

4.3.3 Node Flow Graph

Node Flow Graph - OrderHandler.GetMyOrders



4.3.4 Cyclomatic Complexity

$$V(G) = P + 1 = 3 + 1 = 4$$

#	Predicate Node	Kondisi
P1	Node 3	!exists — user_id tidak ada di context JWT
P2	Node 6	!ok — user_id ada tapi bukan tipe string
P3	Node 10	err != nil — usecase/repository mengembalikan error

Tabel 9. Predicate Node – OrderHandler.GetMyOrders

4.3.5 Independent Path dan Test Case

ID	BB	Path	Skenario	Expected	Status
WB-CHK-G01	CHK-06	G-1	user_id tidak ada	HTTP 401	PASS
WB-CHK-G02	CHK-06	G-2	user_id bukan string	HTTP 401	PASS
WB-CHK-G03	Tambahan	G-3	Usecase/repository error	HTTP 500	PASS
WB-CHK-G04	CHK-08	G-4	Ambil order berhasil	HTTP 200, order PENDING	PASS

Tabel 10. Test Case – OrderHandler.GetMyOrders

4.4 Pengujian Fungsi orderUsecase.Checkout

File: order_usecase.go (baris 68-93)

4.4.1 Deskripsi Fungsi

Business logic layer untuk proses checkout. Memvalidasi cart, memanggil repository ACID transaction, lalu mengintegrasikan dengan Midtrans Snap API untuk pembayaran. Midtrans failure tidak membatalkan order — order tetap dikembalikan, error hanya dicatat melalui log.

4.4.2 Source Code

```
func (u *orderUsecase) Checkout(userID string, voucherCode string)
(*domain.Order, error) {
    cartItems, err := u.cartRepo.FindByUserID(userID)
    if err != nil {
        return nil, errors.New("gagal memuat keranjang belanja")
    }

    if len(cartItems) == 0 {
        return nil, errors.New("keranjang belanja anda kosong.
tidak bisa checkout")
    }
}
```

```

        order, err := u.orderRepo.CheckoutTransaction(userID,
cartItems, voucherCode)
        if err != nil {
            return nil, errors.New("Checkout gagal: " + err.Error())
        }

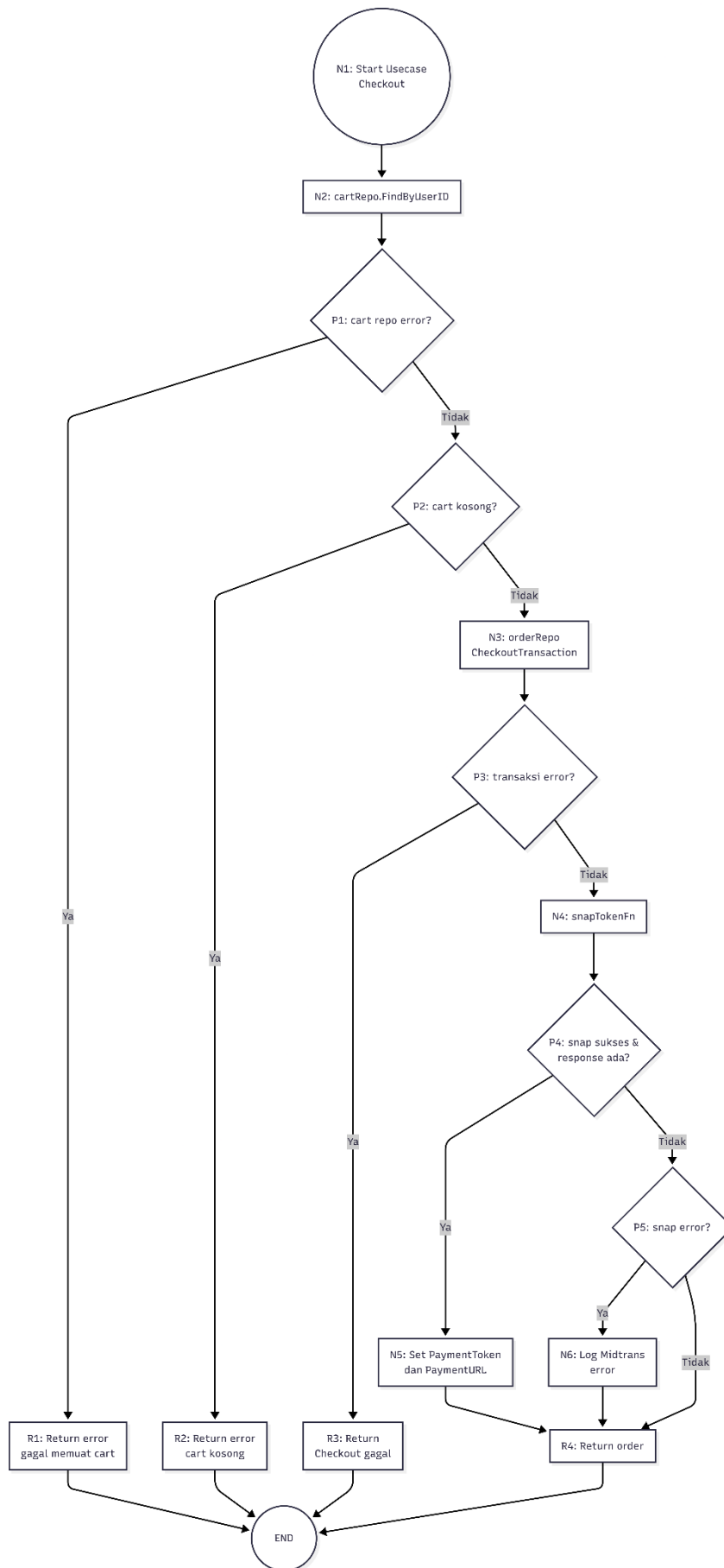
        snapResp, snapErr := u.snapTokenFn(order.ID, order.TotalAmount)
        if snapErr == nil && snapResp != nil {
            order.PaymentToken = &snapResp.Token
            order.PaymentURL = &snapResp.RedirectURL
            u.orderRepo.SavePaymentToken(order.ID, snapResp.Token,
snapResp.RedirectURL)
        } else if snapErr != nil {
            fmt.Printf("[MIDTRANS ERROR] Gagal generate Snap Token:
%v\n", snapErr)
        }

        return order, nil
    }
}

```

4.4.3 Node Flow Graph

Node Flow Graph - orderUsecase.Checkout



4.4.4 Cyclomatic Complexity

$$V(G) = P + 1 = 5 + 1 = 6$$

#	Predicate Node	Kondisi
P1	Node 3	err != nil setelah cartRepo.FindByUserID
P2	Node 5	len(cartItems) == 0
P3	Node 8	err != nil setelah CheckoutTransaction
P4	Node 11	snapErr == nil && snapResp != nil
P5	Node 13	snapErr != nil

Tabel 11. Predicate Node – orderUsecase.Checkout

4.4.5 Independent Path dan Test Case

ID	BB	Path	Skenario	Expected	Status
WB-CHK-U01	Tambahan	U-1	Cart repository error	Error: gagal memuat keranjang	PASS
WB-CHK-U02	CHK-04	U-2	Cart kosong	Error cart kosong	PASS
WB-CHK-U03	CHK-02	U-3	Transaksi gagal (stok)	Error: Checkout gagal: ...stok...	PASS
WB-CHK-U04	CHK-01/08	U-4	Checkout sukses + Snap OK	Order PENDING, token tersimpan	PASS
WB-CHK-U05	CHK-05	U-5	Midtrans gagal	Order dikembalikan, token kosong	PASS
WB-CHK-U06	CHK-07	U-6	Snap response nil tanpa error	Order tetap valid	PASS

Tabel 12. Test Case – orderUsecase.Checkout

4.5 Pengujian Fungsi CheckoutTransaction (Repository ACID)

File: order_repository.go (baris 23-163)

4.5.1 Deskripsi Fungsi

Fungsi inti yang menjamin properti ACID pada proses checkout. Seluruh operasi (pengurangan stok, pembuatan order, penghapusan cart) berjalan dalam satu db.Transaction(). Kegagalan di titik manapun akan menyebabkan ROLLBACK otomatis. Keberhasilan seluruh operasi akan menghasilkan COMMIT otomatis via GORM. Fungsi ini mengimplementasikan SELECT FOR UPDATE (clause.Locking) untuk mencegah race condition pada stok saat checkout bersamaan.

4.5.2 Source Code

```
func (r *orderRepository) CheckoutTransaction(
    userID string, cartItems []domain.CartItem,
    voucherCode string) (*domain.Order, error) {
    var createdOrder domain.Order

    err := r.db.Transaction(func(tx *gorm.DB) error {
        var totalAmount float64
        var orderItems []domain.OrderItem
        orderID := uuid.New().String()

        for _, item := range cartItems {
            var product domain.Product
            if err := tx.Clauses(clause.Locking{Strength:
"UPDATE"}).
                Where("id_product = ?", item.ProductID).
                First(&product).Error; err != nil {
                return errors.New("produk " + item.ProductID + "
tidak ditemukan")
            }
            priceAtPurchase := product.Price

            if item.VariantID != nil {
                var variant domain.ProductVariant
                if err := tx.Clauses(clause.Locking{Strength:
"UPDATE"}).
                    Where("id_variant = ?", *item.VariantID).
                    First(&variant).Error; err != nil {
                    return errors.New("varian produk tidak
ditemukan")
                }
                if variant.Stock < item.Quantity {
                    return fmt.Errorf("stok varian '%s' tidak
mencukupi. Stok tersisa: %d",
                        variant.NameLabel, variant.Stock)
                }
                variant.Stock -= item.Quantity
                if err := tx.Save(&variant).Error; err != nil {
                    return err
                }
                priceAtPurchase = variant.Price
            } else {
                if product.Stock < item.Quantity {
                    return fmt.Errorf("stok produk '%s' tidak
mencukupi. Stok tersisa: %d",
                        product.Name, product.Stock)
                }
            }
        }
        createdOrder = &domain.Order{
            UserID: userID,
            VoucherCode: voucherCode,
            OrderID: orderID,
            TotalAmount: totalAmount,
            OrderItems: orderItems,
        }
    })
    return &createdOrder, err
}
```

```

        }
        product.Stock -= item.Quantity
        if err := tx.Save(&product).Error; err != nil {
return err }
    }
    totalAmount += priceAtPurchase * float64(item.Quantity)
    orderItems = append(orderItems, domain.OrderItem{...})
}

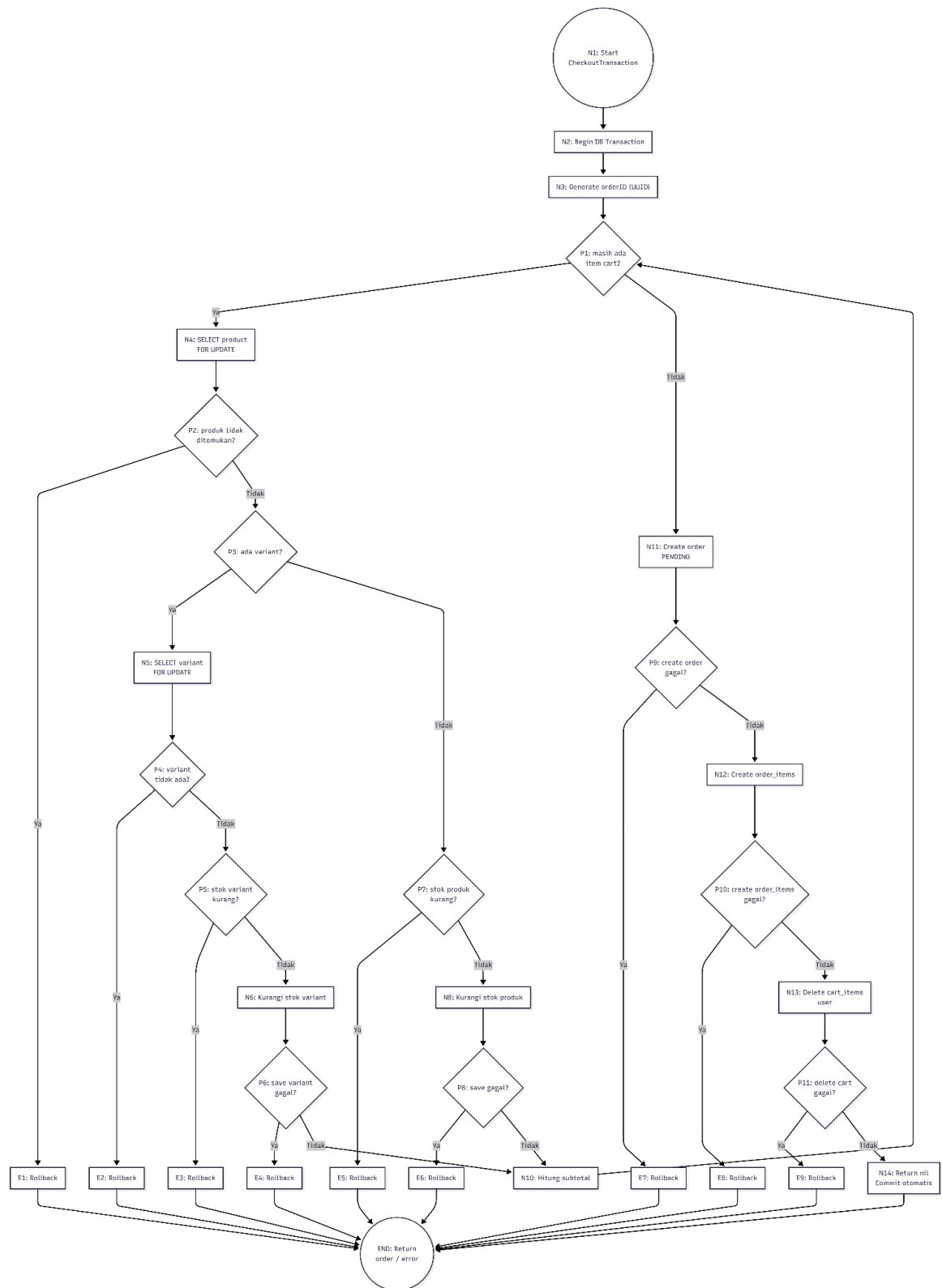
    createdOrder = domain.Order{
        ID: orderID, UserID: userID,
        TotalAmount: totalAmount, Status: "PENDING",
    }
    if err := tx.Create(&createdOrder).Error; err != nil {
return err }
    if err := tx.Create(&orderItems).Error; err != nil { return
err }
    if err := tx.Where("id_user = ?", userID).
        Delete(&domain.CartItem{}).Error; err != nil { return
err }
    return nil
})

    if err != nil { return nil, err }
    return &createdOrder, nil
}

```

4.5.3 Node Flow Graph

Node Flow Graph – CheckoutTransaction (ACID)



4.5.4 Cyclomatic Complexity

Scope aktif tidak menyertakan path voucher (tidak digunakan). $V(G)$ dihitung berdasarkan predicate aktif:

$$V(G) = P + 1 = 11 + 1 = 12 \quad | \quad \text{Kategori Risiko: Sedang}$$

P	Predicate	Kondisi
P1	Loop item cart	Masih ada item yang belum diproses
P2	Produk tidak ditemukan	err != nil setelah SELECT FOR UPDATE produk
P3	Item memiliki variant	item.VariantID != nil
P4	Variant tidak ditemukan	err != nil setelah SELECT FOR UPDATE variant
P5	Stok variant kurang	variant.Stock < item.Quantity
P6	Gagal Save variant	err != nil setelah tx.Save(&variant)
P7	Stok produk kurang	product.Stock < item.Quantity
P8	Gagal Save produk	err != nil setelah tx.Save(&product)
P9	Gagal Create order	err != nil setelah tx.Create(&order)
P10	Gagal Create order_items	err != nil setelah tx.Create(&orderItems)
P11	Gagal Delete cart	err != nil setelah tx.Delete(&CartItem)

Tabel 13. Predicate Node – CheckoutTransaction

4.5.5 Independent Path dan Test Case

ID	BB	Path	Skenario	Expected	Status
WB-CHK-R01	Tambahan	R-01	Produk tidak ditemukan	Error, rollback	PASS
WB-CHK-R02	Tambahan	R-02	Variant tidak ditemukan	Error, rollback	PASS
WB-CHK-R03	CHK-02	R-03	Stok variant tidak cukup	Error, rollback	PASS
WB-CHK-R04	Tambahan	R-04	DB error saat save variant	Rollback	PASS
WB-CHK-R05	CHK-02	R-05	Stok produk utama tidak cukup	Error, rollback	PASS
WB-CHK-R06	Tambahan	R-06	DB error saat save produk	Rollback	PASS
WB-CHK-R07	Tambahan	R-07	DB error saat create order	Rollback	PASS
WB-CHK-R08	Tambahan	R-08	DB error create order items	Rollback	PASS
WB-CHK-R09	Tambahan	R-09	DB error saat delete cart	Rollback	PASS
WB-CHK-R10	CHK-01/07	R-10	Checkout produk non-variant berhasil	Commit — stok -qty, cart=0, PENDING	PASS

ID	BB	Path	Skenario	Expected	Status
WB-CHK-R11	CHK-01/07	R-11	Checkout produk variant berhasil	Commit — stok variant -qty, cart=0	PASS
WB-CHK-R12	CHK-03	R-12	Checkout concurrent stok terbatas	Tidak overselling	PASS

Tabel 14. Test Case – CheckoutTransaction

BAB V BAB V REKAPITULASI DAN KESIMPULAN

5.1 Rekapitulasi Cyclomatic Complexity

No	Fungsi	File	P	V(G)	Risiko
1	classifyCheckoutError	order_handler.go	3	4	Rendah
2	OrderHandler.Checkout	order_handler.go	3	4	Rendah
3	OrderHandler.GetMyOrders	order_handler.go	3	4	Rendah
4	orderUsecase.Checkout	order_usecase.go	5	6	Rendah
5	CheckoutTransaction	order_repository.go	11	12	Sedang
	Total Scope Aktif (5 Fungsi)		25	30	

Tabel 15. Rekapitulasi Cyclomatic Complexity

5.2 Mapping White-Box ke Black-Box

ID BB	Skenario	White-Box Path	Status
CHK-01	Checkout berhasil	H-4, U-4, R-10, R-11	PASS
CHK-02	Stok tidak mencukupi	CE-2, H-3, U-3, R-03, R-05	PASS
CHK-03	Race condition checkout	R-12	PASS
CHK-04	Keranjang kosong	CE-1, H-3, U-2	PASS
CHK-05	Midtrans API error	CE-3, U-5	PASS
CHK-06	Checkout tanpa JWT/session	H-1, H-2, G-1, G-2	PASS
CHK-07	Atomicity stok	R-03, R-05, R-10, R-11	PASS
CHK-08	Status pesanan PENDING	U-4, G-4, R-10, R-11	PASS

Tabel 16. Mapping White-Box ke Black-Box

5.3 Hasil Eksekusi Pengujian

5.3.1 Ringkasan Hasil

Fungsi	Path Aktif	PASS	FAIL	%
classifyCheckoutError	4	4	0	100%
OrderHandler.Checkout	4	4	0	100%
OrderHandler.GetMyOrders	4	4	0	100%
orderUsecase.Checkout	6	6	0	100%
CheckoutTransaction	12	12	0	100%
TOTAL SCOPE AKTIF	30	30	0	100%

Tabel 17. Ringkasan Hasil Pengujian

Coverage path aktif: 30/30 = 100%

Jumlah FAIL: 0

5.3.2 Perintah Verifikasi

```
cd c:\xampp\htdocs\e-commerce-sqa\backend-go
go test ./internal/delivery/http ./internal/usecase
./internal/repository -v
```

Hasil verifikasi terakhir:

```
ok  github.com/nuryanfa/e-commerce-sqa/internal/delivery/http
ok  github.com/nuryanfa/e-commerce-sqa/internal/usecase
ok  github.com/nuryanfa/e-commerce-sqa/internal/repository
```

5.3.3 File Test yang Digunakan

File	Cakupan
order_handler_test.go	classifyCheckoutError
order_handler_checkout_test.go	OrderHandler.Checkout, GetMyOrders
order_usecase_test.go	orderUsecase.Checkout, usecase GetMyOrders
order_repository_test.go	Path repository dengan mock controllable
order_repository_sqlmock_test.go	DB failure path dan rollback dengan sqlmock

Tabel 18. File Test yang Digunakan

5.4 Kesimpulan

Berdasarkan pengujian White Box dengan metode Basic Path Testing yang telah dilaksanakan, dapat disimpulkan hal-hal sebagai berikut:

1. Seluruh independent path aktif pada modul Checkout ACID telah berhasil diidentifikasi dan diuji. Total terdapat 30 independent path aktif yang tersebar pada 5 fungsi utama.
2. Seluruh 30 path aktif dinyatakan LULUS (PASS) pengujian, dengan coverage path aktif mencapai 100% (30/30).
3. Fungsi CheckoutTransaction dengan $V(G) = 12$ merupakan fungsi dengan kompleksitas tertinggi (kategori Sedang) dan memerlukan paling banyak test case (12 path). Namun demikian, seluruh path berhasil diuji dan lulus.
4. Implementasi properti ACID pada CheckoutTransaction terbukti berjalan dengan benar: setiap kegagalan di titik manapun dalam transaksi

menghasilkan ROLLBACK otomatis, dan keberhasilan seluruh operasi menghasilkan COMMIT yang tepat.

5. Pemetaan test case White Box ke skenario Black Box (CHK-01 hingga CHK-08) menunjukkan konsistensi dan saling melengkapi antara kedua pendekatan pengujian.

Metrik	Hasil
Total path aktif	30
PASS	30
FAIL	0
Coverage path aktif	100%

VERDICT FINAL: Modul Checkout ACID LULUS pengujian White Box Basic Path Testing untuk scope aktif. Fitur voucher tidak dimasukkan dalam scope pengujian karena tidak digunakan pada implementasi aktif.

DAFTAR PUSTAKA

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320. <https://doi.org/10.1109/TSE.1976.233837>

Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3rd ed.). John Wiley & Sons.

GORM. (2024). GORM Guides – Transactions. <https://gorm.io/docs/transactions.html>

Gin Web Framework. (2024). Gin HTTP web framework documentation. <https://gin-gonic.com/docs/>

Midtrans. (2024). Midtrans Snap API Documentation. <https://docs.midtrans.com/reference/snap-api>

The Go Programming Language. (2024). Testing package documentation. <https://pkg.go.dev/testing>

Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.

IEEE Standard 829. (2008). *IEEE Standard for Software and System Test Documentation*. Institute of Electrical and Electronics Engineers.

Jorgensen, P. C. (2014). *Software Testing: A Craftsman's Approach* (4th ed.). CRC Press.